



Storage Management I/O Systems

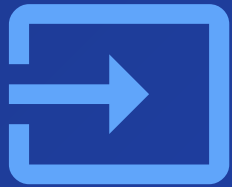
Mata Kuliah Sistem Operasi

Dosen Pengampu

Triawan Adi Cahyanto, M.Kom

Program Studi Teknik Informatika
Universitas Muhammadiyah Jember

2025



What is I/O Systems?

Pengertian I/O Systems

I/O adalah singkatan dari **Input/Output**, yang merupakan cara komputer berkomunikasi dengan dunia luar.

Contoh Perangkat I/O:



Keyboard



Mouse



Monitor



Hard Disk



Printer



Speaker

▮ Tanpa I/O Systems, komputer tidak bisa berinteraksi dengan pengguna maupun perangkat eksternal lainnya!

Pentingnya I/O Systems dalam Sistem Operasi



Komunikasi

Menghubungkan CPU dengan perangkat eksternal seperti keyboard, mouse, monitor, dan hard disk



Manajemen Data

Mengelola data keluar-masuk (input/output) secara efisien dan terstruktur



Efisiensi Sistem

Memastikan sistem operasi bekerja optimal dengan pengelolaan I/O yang tepat



Tanpa I/O Systems yang baik, komputer tidak dapat berinteraksi dengan dunia luar dan sistem operasi akan berjalan dengan sangat lambat!

I/O Hardware - Jenis-jenis Perangkat I/O



Perangkat Input

- Keyboard
- Mouse
- Scanner
- Microphone
- Webcam

Menerima data dari pengguna



Perangkat Output

- Monitor
- Printer
- Speaker
- Projector
- Headphone

Menampilkan hasil proses



Perangkat Penyimpanan

- Hard Disk
- SSD
- USB Flashdisk
- SD Card
- Tape Drive

Menyimpan data secara permanen

I/O Hardware - Cara Kerja Perangkat I/O

1

CPU Mengirim Perintah

CPU mengirimkan perintah I/O ke **controller I/O**

2

Controller Berkomunikasi

Controller I/O berkomunikasi langsung dengan **perangkat I/O**

3

Transfer Data

Data ditransfer melalui **bus sistem** antara CPU dan perangkat I/O

4

Interrupt Notifikasi

Perangkat mengirim **interrupt** untuk memberitahukan CPU bahwa proses selesai



CPU

Perintah I/O

Menerima Interrupt



CPU dapat melanjutkan tugas lain saat menunggu I/O selesai berkat mekanisme interrupt!

Application I/O Interface - Pengertian dan Fungsi



Pengertian

Application I/O Interface adalah **jembatan** yang menghubungkan aplikasi dengan perangkat I/O fisik



Intinya...

Programmer tidak perlu pusing dengan detail hardware, cukup gunakan fungsi standar yang sudah disediakan!



Fungsi Utama

1

API Standar

Menyediakan fungsi standar untuk akses I/O yang konsisten

2

Abstraksi

Menyembunyikan kompleksitas hardware dari programmer

3

Portabilitas

Aplikasi dapat berjalan di berbagai perangkat tanpa modifikasi

4

Efisiensi

Mempermudah pengembangan aplikasi dengan fungsi siap pakai

Application I/O Interface - Contoh Implementasi

<> Konsep

Programmer menggunakan fungsi standar tanpa perlu mengerti detail hardware

Sistem Operasi menerjemahkan fungsi standar menjadi perintah khusus untuk setiap perangkat I/O

Bahasa C

- `fopen()` - Buka file
- `fread()` - Baca data
- `fwrite()` - Tulis data
- `fclose()` - Tutup file

Bahasa Python

- `open()` - Buka file
- `read()` - Baca data
- `write()` - Tulis data
- `close()` - Tutup file



Fungsi yang sama dapat digunakan untuk berbagai perangkat I/O: hard disk, SSD, USB, dll.

Kernel I/O Subsystem - Komponen



Buffering

Menyimpan data sementara saat transfer antara perangkat dengan memori



Caching

Menyimpan data yang sering diakses agar lebih cepat diambil kembali



Spooling

Menangani output printer dengan menyimpan data di memori terlebih dahulu



Device Driver

Berfungsi sebagai penterjemah antara sistem operasi dengan perangkat hardware



Error Handling

Mendeteksi dan menangani kesalahan yang terjadi saat operasi I/O

Kernel I/O Subsystem - Fungsi dan Cara Kerja

⚙️ Fungsi Utama

- ➔ **Koordinasi I/O**
Mengelola semua permintaan I/O dari aplikasi
- ➔ **Optimasi Performa**
Meningkatkan kecepatan akses data
- ➔ **Abstraksi Hardware**
Menyembunyikan kompleksitas perangkat I/O

🔧 Cara Kerja

1

Aplikasi mengirim permintaan I/O ke sistem operasi



2

Kernel I/O Subsystem memproses permintaan

Buffer

Cache

Spool

Driver

Error



3

Perintah dikirim ke perangkat I/O

Transforming I/O Requests - Proses Transformasi

Dari Permintaan Aplikasi ke Perintah Hardware

1

<> Aplikasi

Menggunakan fungsi standar

`fread(), fwrite()`

2

OS

Menerjemahkan permintaan

Kernel I/O Subsystem

3

Driver

Memanggil device driver

Berhubungan dengan hardware

4

Hardware

Mengirim perintah ke I/O

Controller I/O

Level Tinggi



Menengah



Level Rendah



Hardware

Transforming I/O Requests - Langkah-langkah dan Contoh

Contoh Kasus

Aplikasi ingin membaca file text dari hard disk

Aplikasi

Memanggil fungsi standar

```
fread(file, buffer, size)
```

Sistem Operasi

Menerjemahkan permintaan I/O

Proses Transformasi

1

Kernel I/O mengecek cache memori



2

Jika tidak ada di cache, panggil disk driver



3

Disk driver membaca sektor dari hard disk



4

Data ditransfer ke buffer aplikasi

i

Seluruh proses berlangsung tanpa programmer perlu memahami detail hardware

Streams - Pengertian dan Jenis-jenis

Apa itu Streams?

Representasi **abstrak** dari sumber data atau tujuan data

Aliran data yang masuk atau keluar dari program

Memudahkan pemrograman I/O dengan pendekatan yang **terstruktur**

Jenis-jenis Streams

Input Stream

Membaca data dari sumber

Keyboard, file, jaringan

Output Stream

Menulis data ke tujuan

Monitor, printer, file

Byte Stream

Data dalam bentuk byte

Binary data, gambar, audio

Character Stream

Data dalam bentuk karakter

Teks, string, dokumen

Streams - Penggunaan dalam I/O

Mengapa Streams?

- ✓ Antarmuka **konsisten** untuk berbagai jenis I/O
- ✓ Kode lebih **reusable** dan mudah dimengerti
- ✓ Memisahkan logika bisnis dari detail I/O

<> Contoh Penggunaan Streams

FileInputStream

Membaca file

```
new  
FileInputStream("file")
```

BufferedReader

Baca teks efisien

```
new  
BufferedReader(...)
```

PrintStream

Tulis output

```
System.out.println()
```

FileOutputStream

Tulis file

```
new  
FileOutputStream("fil")
```

InputStreamReader

Baca keyboard

```
new  
InputStreamReader(Sys
```

NetworkStream

I/O jaringan

```
Socket.getInputStream()
```

 Semua jenis I/O dapat ditangani dengan pendekatan yang sama menggunakan streams

Performance - Faktor dan Optimasi

Faktor-Faktor Performa



Kecepatan Perangkat

SSD 10x lebih cepat dari HDD



Ukuran Transfer

Transfer besar lebih efisien



Caching

Menyimpan data yang sering diakses



Buffering

Menyimpan data sementara

Strategi Optimasi

✓ Gunakan Cache Memori

Menyimpan data yang sering diakses di memori yang cepat

✓ Implementasi Buffering

Mengurangi jumlah operasi I/O dengan menumpuk data

✓ Pilih Perangkat Tepat

Gunakan SSD untuk database, HDD untuk backup



Optimasi I/O dapat meningkatkan performa aplikasi secara signifikan

Summary



I/O Systems

→ Cara komputer berkomunikasi dengan perangkat eksternal

Perangkat Input, Output, Penyimpanan

Application I/O Interface

Jembatan antara aplikasi dan hardware

Kernel I/O Subsystem

Buffering, Caching, Spooling, Driver, Error Handling

Transforming I/O Requests

Dari aplikasi level tinggi ke hardware level rendah

Streams

Representasi abstrak sumber dan tujuan data

Performance

Kecepatan perangkat

Caching & Buffering

Optimasi I/O

Komponen Utama

Hardware, Interface, Kernel, Streams

Tujuan

Efisiensi komunikasi data